

QUANT FINANCE TERMINAL ·
v2.046

SYS://CURRICULUM/D056.MD · PHASE 2 ●
LIVE

LECTURE · 量化金融 365

DAY 056 / 365

P2 · PYTHON 量化工具栈

DEEP DIVE · 8 页深度版

单元测试与策略

// Unit Tests

KEY CONCEPTS · 三大要点

- ▶ pytest
- ▶ fixture
- ▶ 参数化

01 · WHY THIS LESSON · 这节课为什么重要

本节定调

// 看完这一段你会知道:这节课跟你接下来 3 个月的学习节奏关系有多大

做量化最怕的不是行情看错,而是代码悄悄算错了你还不知道。一个指标算错符号、一段回测藏着 bug,跑出来的数字照样漂漂亮亮,你却照着这堆错数字下单,亏的是真金白银。这一节请出工程师的护身符:单元测试。它就像给你的每个函数出一套标准考题,你一改代码,就把所有考题重新做一遍,谁没考过立刻报警。我们用 pytest 这个工具,给几个常用指标函数(均线、最大回撤、相对强弱)写考题,故意埋一个算错符号的 bug 版本,看看测试能不能一把把它揪出来。学完这节,你改代码时心里会踏实很多,因为有一群考题在替你盯着。

学习目标 · LEARNING OBJECTIVES

// 听课前默念一遍,听完之后回头打勾

- 01 理解单元测试是什么:给每个函数出一套标准考题,一改代码就全部重考,谁错谁报警
- 02 会用 pytest 这个工具,给指标函数写测试用例,一条命令把所有考题跑一遍
- 03 会用 assert 断言写考题答案、用边界用例覆盖空数据和极端值这些最容易出 bug 的角落
- 04 会用 fixture 复用测试数据、用 parametrize 一道题套多组输入,少写重复代码
- 05 明白量化代码不写测试的代价:指标算错、回测有 bug,直接亏真金白银

02 · CONTEXT · 历史与背景

老蒋把回撤算反了符号,照着错指标选股亏了一笔

// 不读历史的策略走不远

老蒋是个会写 python 的散户,自己撸了一套选股程序,其中有个算最大回撤的函数,用来挑那些跌得不狠、相对抗跌的票。问题是他手滑,把回撤的符号写反了:本该是越接近零越抗跌,他的函数却把跌得最狠的票算成了分数最高。程序跑得顺顺当当,没有任何报错,打印出来的数字也像模像样,他压根没发现。于是他满心欢喜地照着这个排名,重仓买进了几只他以为最抗跌、其实最脆弱的票。结果一波回调下来,这几只跌得比谁都惨,他被结结实实割了一刀,还百思不得其解:我的逻辑明明没问题啊。后来他痛定思痛,给每个指标函数补了单元测试,专门拿几组手算好答案的小数据当考题。再后来某次他改代码,不小心又碰坏了一个地方,考题一跑,立刻有一条标红报警,他当场就发现并改回来了,再没栽过这种暗坑。老蒋的教训很实在:量化代码出错往往不吭声,你得主动给它出考题,逼它现原形。

把这几个概念彻底搞懂

// 听完视频后,确保你能给一个完全不懂的朋友讲清楚每一条

CONCEPT_01

单元测试:给每个函数出一套标准考题

单元测试就是给你的每个小函数,准备一套答案已知的标准考题。比如你写了个加法函数,你就出一道题:输入 2 和 3,正确答案应该是 5。然后让程序自动做这道题,对上 5 就算通过,对不上就报警。你一改代码,就把所有考题重做一遍,只要哪个函数被改坏了,对应的考题立刻考不过、马上报红。它最大的价值是把『改坏了却没发现』这种暗坑挡在门外。

CONCEPT_02

assert 断言:写考题的标准答案

assert 是写考题答案的关键词,意思是『我断定这件事是对的,如果不对就报警』。比如 `assert add(2, 3) == 5`,翻译成大白话就是『我断定 2 加 3 等于 5,要是不等就给我报错』。程序跑到这一行,条件成立就悄悄通过、继续往下;条件不成立就立刻抛错停下来,告诉你这道考题没过。一个测试里可以写好几条 assert,把一个函数的多个方面都考一遍。

03 · CORE CONCEPTS · 续 (2/3)

CONCEPT_03

fixture:复用的测试数据

很多考题都要用到同一份样本数据,比如同一段长江电力的收盘价。要是每道题都重新准备一遍数据,又啰嗦又容易写错。fixture 就是把这份公共数据准备好放在一处,哪道考题需要就直接领用。比如你做个 fixture 返回一段固定的价格序列,五道考题都用它当输入,数据只写一遍。它让测试代码干净、不重复,改数据也只改一个地方。

CONCEPT_04

parametrize 参数化:一道题套多组输入

有时候同一道考题,你想用好几组不同的输入分别试一遍。比如测均线函数,你想试窗口为 2、3、5 三种情况。笨办法是复制粘贴写三个几乎一样的测试,既累又难维护。parametrize 参数化就是让你把『多组输入和对应的正确答案』列成一张表,pytest 自动拿每一组各跑一遍,等于一道题瞬间变成好几道题。代码只写一份,覆盖面却成倍扩大。

CONCEPT_05

边界用例:空数据和极端值最容易出 bug

考题不能只考正常情况,真正容易翻车的是边边角角:空数据(一个数都没有)、只有一个数、所有数都一模一样、特别大或特别小的极端值。这些边界情况你平时想不到,代码常常没处理好,一遇到就崩或者算错。老蒋的 bug 就藏在他没考虑到的角落里。所以写考题时一定要专门出几道边界题,把这些极端角落都考一遍,bug 往往就栽在这里。

04 · PYTHON LAB · 真实可跑代码

pytest 单元测试 — assert 断言 / fixture 复用数据 / parametrize 参数 化 / 正确版全过 vs bug 版被揪出

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

► **实操原则:** 先跑通(哪怕硬编码),再优化(参数化/函数化),最后才追求精度(模型升级/特征工程)。散户量化最大的坑是没跑通就开始优化。

```
# day_056_pytest_unit_test.py - 用 pytest 给量化指标函数写单元测试
# 真名上屏:pytest / assert 断言 / @pytest.fixture 复用数据 / @pytest.mark.parametrize
# 参数化
# 核心:给每个函数出标准考题,一改代码就全部重考;故意埋一个 bug 版本,看测试能不能把它揪出来
import os
import sys
import subprocess
import textwrap
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import baostock as bs

def _data_path(_name):
    # 铁律62:data/ 放在 notebook 文件夹里。仓库根(run_lab)存取 out/notebook/data/;
    # 原版 notebook 在 out/notebook/ 用 data/;中国版在 out/notebook/cn/ 用 ../data/
    from pathlib import Path as _P
    _here = _P.cwd()
    for _b in [_here/'data', _here/'../'/'data', _here/'out'/'notebook'/'data',
               _here/'../'/'../'/'data', _here/'../'/'../'/'../'/'data']:
        if (_b/_name).exists():
            return str(_b/_name)
    if (_here/'out'/'notebook').exists():
        _t = _here/'out'/'notebook'/'data'
    elif _here.name == 'cn':
```

04 · PYTHON LAB · 真实可跑代码 · 续 (2/9)

pytest 单元测试 — assert 断言 / fixture 复用数据 / parametrize 参数 化 / 正确版全过 vs bug 版被揪出

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
_t = _here/ '../data'
else:
    _t = _here/ 'data'
_t.mkdir(parents=True, exist_ok=True)
return str(_t/_name)

pd.set_option('display.width', 160)
plt.rcParams['axes.unicode_minus'] = False
CODE, NAME = 'sh.600900', '长江电力'
START, END = '2021-01-01', '2024-12-31'
CSV_PATH = _data_path('D056_pytest.csv')

# ==== 0. 拉长江电力日线收盘,作为指标函数的真实输入示例 ====
print('==== 0. 拉长江电力日线收盘,作为被测函数的真实输入 ====')
if os.path.exists(CSV_PATH):
    # 缓存优先:CSV 在就直接读,不联网
    px = pd.read_csv(CSV_PATH, parse_dates=['date'])
else:
    lg = bs.login()
    if lg.error_code != '0':
        raise RuntimeError(f'baostock 登录失败:{lg.error_msg}')
    rs = bs.query_history_k_data_plus(CODE, 'date,close', start_date=START,
                                     end_date=END, frequency='d')
    rows = []
    while rs.error_code == '0' and rs.next():
        rows.append(rs.get_row_data())
    bs.logout()
    px = pd.DataFrame(rows, columns=['date', 'close'])
    px.to_csv(CSV_PATH, index=False) # 拉完保存 CSV
    px['date'] = pd.to_datetime(px['date'])
```

04 · PYTHON LAB · 真实可跑代码 · 续 (3/9)

pytest 单元测试 — assert 断言 / fixture 复用数据 / parametrize 参数 化 / 正确版全过 vs bug 版被揪出

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
px['close'] = pd.to_numeric(px['close'])
px = px.set_index('date').sort_index()
closes = px['close'].to_numpy()
print(f'{NAME} 共 {len(closes)} 个交易日的收盘价,这就是要喂给指标函数的真实数据')

# ==== 1. 被测的指标函数(纯函数):正确版 + 故意埋 bug 的版本 ====
print('\n==== 1. 三个指标函数:均线 / 最大回撤 / 相对强弱(各一正确版) ====')

def moving_average(arr, n):
    """简单均线:返回长度为 len(arr)-n+1 的均值序列;空数据或窗口非法返回空数组"""
    a = np.asarray(arr, dtype=float)
    if a.size == 0 or n <= 0 or n > a.size:
        return np.array([])
    c = np.cumsum(np.insert(a, 0, 0.0))
    return (c[n:] - c[:-n]) / n

def max_drawdown(equity):
    """最大回撤(正确版):返回 <=0 的负数,越接近 0 越抗跌;空/单值返回 0.0"""
    e = np.asarray(equity, dtype=float)
    if e.size < 2:
        return 0.0
    peak = np.maximum.accumulate(e)
    dd = (e - peak) / peak
    return float(dd.min())

def max_drawdown_buggy(equity):
    """最大回撤(故意埋 bug 版):把符号写反 + 没处理空数组,演示测试能否揪出来"""
    e = np.asarray(equity, dtype=float)
    peak = np.maximum.accumulate(e)
    dd = (e - peak) / peak
```

04 · PYTHON LAB · 真实可跑代码 · 续 (4/9)

pytest 单元测试 — assert 断言 / fixture 复用数据 / parametrize 参数 化 / 正确版全过 vs bug 版被揪出

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
return float(dd.max()) # bug: 该取min 却取了max,符号搞反

def rsi(arr, n):
    """相对强弱RSI:返回 0~100 的数;数据不足或全相同时返回中性值 50.0"""
    a = np.asarray(arr, dtype=float)
    if a.size < n + 1:
        return 50.0
    diff = np.diff(a)
    gain = diff[diff > 0].sum()
    loss = -diff[diff < 0].sum()
    if gain + loss == 0:
        return 50.0
    return float(100.0 * gain / (gain + loss))

print('moving_average / max_drawdown / rsi 已定义;另有一个故意写反符号的
max_drawdown_buggy 备考')

# ==== 2. 用手算小数据先验证一下正确版(还没用pytest)====
print('\n==== 2. 先用手算小例子摸一下底 ====')
print(f'均线 [1,2,3,4] 窗口2 = {moving_average([1,2,3,4], 2)} (手算应是
[1.5,2.5,3.5])')
print(f'回撤正确版 [100,120,90,110] = {max_drawdown([100,120,90,110]):.4f} (应为负
数,约 -0.25)')
print(f'回撤bug 版 [100,120,90,110] = {max_drawdown_buggy([100,120,90,110]):.4f}
(符号反了,成了非负)')
print(f'真实数据上长江电力最大回撤 = {max_drawdown(closes):.4f} (越接近0越抗跌)')

# ==== 3. 把一整套pytest 考题写到临时文件 test_indicators.py ====
print('\n==== 3. 生成pytest 测试文件:assert / fixture / parametrize 全用上 ====')
TEST_FILE = os.path.join('/tmp', 'test_indicators.py')
test_src = textwrap.dedent(''
```

```
import numpy as np
import pytest
from indicators_under_test import moving_average, max_drawdown, rsi
```

04 · PYTHON LAB · 真实可跑代码 · 续 (5/9)

pytest 单元测试 — assert 断言 / fixture 复用数据 / parametrize 参数 化 / 正确版全过 vs bug 版被揪出

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
# ---- fixture:复用的样本数据,哪道考题需要就直接领用 ----
@pytest.fixture
def sample_equity():
    return [100.0, 120.0, 90.0, 110.0, 80.0, 130.0]

# ---- 正常用例:用 assert 写考题的标准答案 ----
def test_moving_average_normal():
    out = moving_average([1, 2, 3, 4], 2)
    assert np.allclose(out, [1.5, 2.5, 3.5])

def test_max_drawdown_is_negative(sample_equity):
    dd = max_drawdown(sample_equity)
    assert dd <= 0 # 回撤必须是非正数,越接近0越抗跌
    assert dd == pytest.approx(-0.3333, abs=1e-3)

def test_rsi_in_range():
    val = rsi([1, 2, 3, 2, 3, 4, 5], 3)
    assert 0 <= val <= 100 # RSI 永远落在 0 到 100 之间

# ---- 边界用例:空数据 / 单个值 / 全相同,最容易出 bug 的角落 ----
def test_moving_average_empty():
    assert moving_average([], 3).size == 0

def test_max_drawdown_single_value():
    assert max_drawdown([100.0]) == 0.0

def test_rsi_all_same():
    assert rsi([5, 5, 5, 5, 5], 3) == 50.0 # 全相同时给中性值
```

04 · PYTHON LAB · 真实可跑代码 · 续 (6/9)

pytest 单元测试 — assert 断言 / fixture 复用数据 / parametrize 参数 化 / 正确版全过 vs bug 版被揪出

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
# ---- parametrize 参数化:一道题套多组输入,自动各跑一遍 ----
@pytest.mark.parametrize('arr,n,expected', [
    ([1, 2, 3], 1, [1, 2, 3]),
    ([1, 2, 3, 4], 2, [1.5, 2.5, 3.5]),
    ([2, 4, 6, 8], 4, [5.0]),
])
def test_moving_average_params(arr, n, expected):
    assert np.allclose(moving_average(arr, n), expected)
    '''

with open(TEST_FILE, 'w', encoding='utf-8') as f:
    f.write(test_src)
print(f'考题已写入 {TEST_FILE}:含正常用例 / 空数据等边界用例 / parametrize 多组输入 /
fixture 样本')

# ==== 4. 程序化跑 pytest:先跑正确版(应全过),再跑 bug 版(应有 fail 被揪出)====
print('\n==== 4. 跑两次 pytest:正确版 vs 故意带 bug 版 ====')

def write_under_test_module(use_buggy):
    """把被测函数写到 indicators_under_test.py;use_buggy=True 时换成符号写反的回撤"""
    dd_impl = 'def max_drawdown(e):\n e=np.asarray(e,float)\n
    peak=np.maximum.accumulate(e)\n return float(((e-peak)/peak).max())\n'
    if not use_buggy:
        dd_impl = 'def max_drawdown(e):\n e=np.asarray(e,float)\n if e.size<2: return
        0.0\n peak=np.maximum.accumulate(e)\n return float(((e-peak)/peak).min())\n'
    mod = textwrap.dedent(''
    import numpy as np
    def moving_average(arr, n):
        a=np.asarray(arr,float)
        if a.size==0 or n<=0 or n>a.size: return np.array([])
        c=np.cumsum(np.insert(a,0,0.0))
        return (c[n:]-c[:-n])/n
    def rsi(arr, n):
        a=np.asarray(arr,float)
```

04 · PYTHON LAB · 真实可跑代码 · 续 (7/9)

pytest 单元测试 — assert 断言 / fixture 复用数据 / parametrize 参数 化 / 正确版全过 vs bug 版被揪出

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
if a.size<n+1: return 50.0
d=np.diff(a); g=d[d>0].sum(); l=-d[d<0].sum()
return 50.0 if g+l==0 else float(100.0*g/(g+l))
''' + dd_impl
with open(os.path.join('/tmp', 'indicators_under_test.py'), 'w',
encoding='utf-8') as f:
f.write(mod)

def run_pytest():
    """用 subprocess 跑 pytest, 返回 (passed, failed, 末尾摘要行)"""
    r = subprocess.run([sys.executable, '-m', 'pytest', TEST_FILE, '-v', '--
tb=short'],
capture_output=True, text=True, cwd='/tmp')
    out = r.stdout + r.stderr
    passed = failed = 0
    summary = ''
    for line in out.splitlines():
        if 'passed' in line or 'failed' in line:
            summary = line.strip()
    import re
    mp = re.search(r'(\d+) passed', out)
    mf = re.search(r'(\d+) failed', out)
    if mp: passed = int(mp.group(1))
    if mf: failed = int(mf.group(1))
    return passed, failed, summary

write_under_test_module(use_buggy=False)
p_ok, f_ok, sum_ok = run_pytest()
print(f'[正确版] 考题结果:{p_ok} 道通过,{f_ok} 道失败 | {sum_ok}')

write_under_test_module(use_buggy=True)
p_bug, f_bug, sum_bug = run_pytest()
```

04 · PYTHON LAB · 真实可跑代码 · 续 (8/9)

pytest 单元测试 — assert 断言 / fixture 复用数据 / parametrize 参数 化 / 正确版全过 vs bug 版被揪出

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
print(f'[bug 版] 考题结果:{p_bug} 道通过,{f_bug} 道失败 | {sum_bug}')
```

```
print('对比:正确版全过,bug 版有考题报红 — 测试这道护栏一把抓出了反符号的 bug')
```



```
# ==== 5. 画三张图 ====
```

```
print('\n==== 5. 画三张图:通过失败对比 / 指标真实输出 / parametrize 覆盖 ====')
```



```
# 图1:正确版 vs bug 版 测试通过/失败 数量对比
```

```
fig, ax = plt.subplots(figsize=(11, 6))
```

```
labels = ['正确版', 'bug 版']
```

```
xpos = np.arange(len(labels))
```

```
ax.bar(xpos - 0.2, [p_ok, p_bug], width=0.4, label='通过', color='#2ca02c')
```

```
ax.bar(xpos + 0.2, [f_ok, f_bug], width=0.4, label='失败(报红)', color='#d62728')
```

```
ax.set_xticks(xpos); ax.set_xticklabels(labels)
```

```
ax.set_title('单元测试结果:正确版全过 vs bug 版被揪出'); ax.set_ylabel('考题数量')
```

```
ax.legend(); ax.grid(alpha=0.3, axis='y'); plt.tight_layout();
```

```
plt.savefig('test_result.png', dpi=120); plt.close()
```



```
# 图2:被测对象具体可见 — 长江电力真实收盘 + 均线 + 最大回撤标注
```

```
ma20 = moving_average(closes, 20)
```

```
fig, ax = plt.subplots(figsize=(11, 6))
```

```
ax.plot(range(len(closes)), closes, color='#9bbbe0', lw=1, label=f'{NAME} 收盘价')
```

```
ax.plot(range(19, 19 + len(ma20)), ma20, color='#ff7f0e', lw=1.5, label='20 日均线
```

```
moving_average')
```

```
mdd = max_drawdown(closes)
```

```
ax.set_title(f'被测函数在真实数据上的输出:{NAME} 收盘 + 均线(最大回撤 {mdd:.1%})')
```

```
ax.set_xlabel('交易日序号'); ax.set_ylabel('价格')
```

```
ax.legend(); ax.grid(alpha=0.3); plt.tight_layout();
```

```
plt.savefig('indicator_output.png', dpi=120); plt.close()
```



```
# 图3:parametrize 多组用例覆盖示意
```

```
cases = [(' [1,2,3] n=1', ' [1,2,3]', True),  
( ' [1,2,3,4] n=2', ' [1.5,2.5,3.5]', True),  
( ' [2,4,6,8] n=4', ' [5.0]', True)]
```

04 · PYTHON LAB · 真实可跑代码 · 续 (9/9)

pytest 单元测试 — assert 断言 / fixture 复用数据 / parametrize 参数 化 / 正确版全过 vs bug 版被揪出

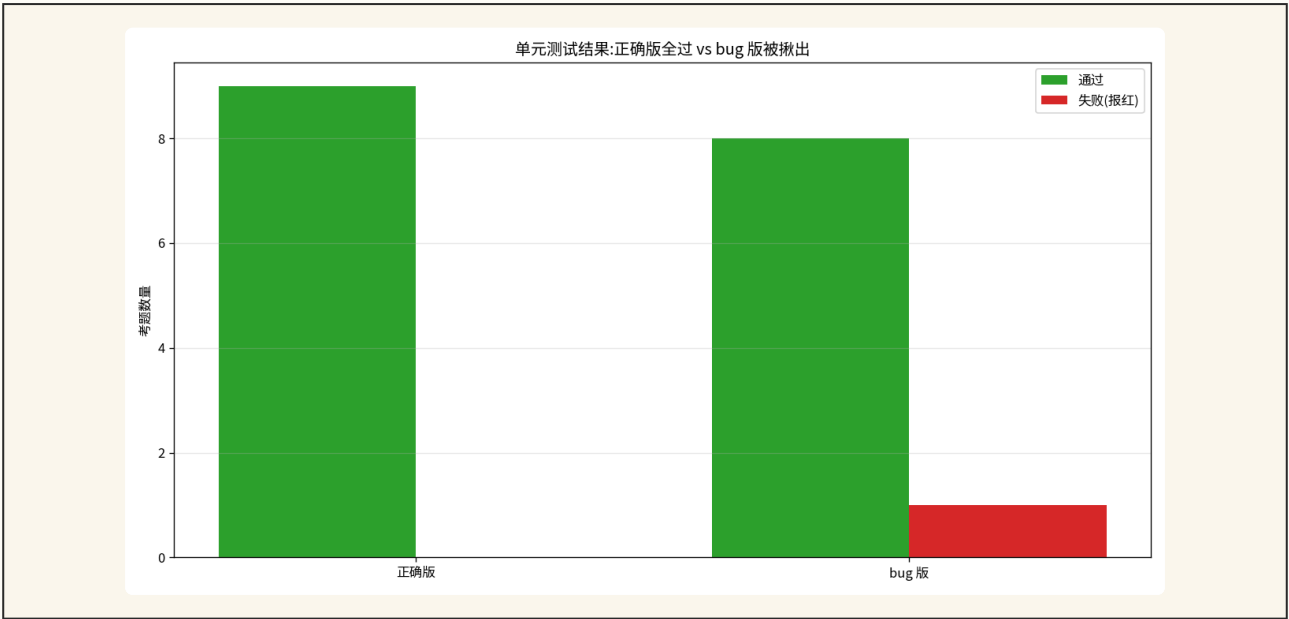
// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
fig, ax = plt.subplots(figsize=(11, 6))
ax.axis('off')
ax.set_title('parametrize:一道题套多组输入,自动各跑一遍', fontsize=14)
tbl_rows = [[c[0], c[1], '通过' if c[2] else '失败'] for c in cases]
table = ax.table(cellText=tbl_rows, colLabels=['输入', '期望输出', '是否通过'],
cellLoc='center', loc='center')
table.auto_set_font_size(False); table.set_fontsize(12); table.scale(1, 2.2)
plt.tight_layout(); plt.savefig('parametrize.png', dpi=120); plt.close()

print('\n[done] 单元测试演示完成:正确版全过 / bug 版被揪出,3 张图已生成')
```

回测结果

// · matplotlib 真实输出



parametrize:一道题套多组输入,自动各跑一遍

输入	期望输出	是否通过
[1,2,3] n=1	[1,2,3]	通过
[1,2,3,4] n=2	[1.5,2.5,3.5]	通过
[2,4,6,8] n=4	[5.0]	通过

被测真实数据(长江电力)

长江电力 969 个交易日的真实收盘价当输入示例;正确版算出最大回撤 -0.2024(从最高点最深跌约两成)

正确版:9 道考题全过

正常用例 + 空数组/单值/全相同等边界用例 + parametrize 多组输入,共 9 道,正确版 9 passed、0 failed,一片绿

bug 版:恰好 1 道报红被抓

只把最大回撤符号写反、其余一字不改,同样 9 道考题里恰好 1 道(回撤那道)failed,其余 8 道照样过——精准定位 bug

手算小例子验证

均线 [1,2,3,4] 窗口 2 = [1.5,2.5,3.5];回撤 [100,120,90,110] 正确版 -0.25、bug 版(符号反)算成 0.00——小例子最能逼出 bug

pytest 三件套

assert 断言写考题标准答案、fixture 复用样本数据只写一遍、parametrize 一道题套多组输入自动各跑一遍——单元测试的核心三件套

边界用例最容易栽

空数组、单个值、全部相同这些边角才是 bug 老巢:正常时跑得好好的,一遇极端输入就崩或算错,考题要专门覆盖

► 01 单元测试 = 给每个函数出一套标准考题

做量化最怕的不是行情看错,而是代码偷偷算错你还不知道:一个指标符号写反、一段回测藏坑,跑出来的数字照样漂亮,你照着下单亏的是真钱。单元测试就是给每个小函数配一套答案已知的考题,比如加法函数出一道『输入 2 和 3 应得 5』,让程序自动做、对上就过、对不上就报警。你一改代码,所有考题重做一遍,哪个函数被改坏,对应考题立刻标红。它最大的本事,就是把『改坏了却没发现』这种暗坑挡在门外。

► 02 正确版 9 道全过 vs bug 版 1 道被揪出

我们给均线、最大回撤、相对强弱三个指标函数写了 9 道考题(正常 + 边界 + 参数化)。先用全部正确的函数跑,9 道全过、0 失败,一片绿。再把一个故意把回撤符号写反的坏版本换进去、其它一字不改,同样 9 道考题里恰好有 1 道标红失败,而且失败的正是考回撤的那道,其余 8 道照样通过。同一堆代码只差一个符号,正确版全过、坏版本被当场抓住——这就是单元测试这道护栏最直接的价值。

► 03 assert / fixture / parametrize 三件套

写考题靠三件工具配合。assert 断言写标准答案:断定某事为真、不真就报错,一道题可连写好几句,比如先断定回撤是非正数、再断定它约等于手算值。fixture 把公共样本数据准备好放一处,哪道考题要用就直接领用,数据只写一遍、改也只改一处。parametrize 参数化把多组输入和对应答案列成一张表,程序自动拿每组各跑一遍,一道题瞬间变好几道、代码只写一份覆盖面成倍扩大。

► 04 边界用例:空数据和极端值才是 bug 老巢

考题不能只考正常情况,真正容易翻车的是边边角角:空数据一个数都没有、只有一个数、所有数一模一样、特别大特别小的极端值。这些角落你平时想不到,代码常常没处理好,一遇到就崩或算错,老蒋的暗坑就藏在他没考虑到的地方。手算小例子也最能逼出问题:回撤 [100,120,90,110] 正确版 -0.25、bug 版符号反成了 0.00,一眼就露馅。所以一定要专门为空数据、单值、极端值各出一道边界题。

► 05 量化代码 bug 直接亏钱,主动出考题逼它现原形

量化代码出错往往不吭声:指标算反、回测藏坑,程序跑得顺顺当当、数字漂漂亮亮,你却照着错数字下大仓位,像老蒋那样把最脆弱的票当成最抗跌的买进,结结实实割一刀。越是真金白银驱动的代码,越值得花点时间出几道考题盯着它。关键不是靠人眼盯输出找毛病——人会犯困会漏,而考题不知疲倦、改一次考一次。代码可以信任,但更要验证,验证越早交给考题,你的账户就越安全。

A 股 · 港股 · 美股 · 加密 全覆盖

// 每个案例都有具体标的和数字,方便你立刻验证

指标函数	sh.600900	给长江电力的均线、最大回撤、RSI 函数写 pytest 考题,用真实收盘价当输入示例
抓 bug		故意埋一个把回撤符号写反的 bug 版本,看单元测试能不能一把把它揪出来
边界覆盖		专门为空数组、单个值、全相同这些极端角落出边界考题,堵住最容易翻车的地方
参数化		用 parametrize 把均线的多组窗口和期望答案列成一张表,一道题自动跑出好几道

这些坑你不主动避 · 迟早会主动找上你

// 每个坑都附了避坑动作,而不是只描述现象

△ 01

只测正常情况,不测边界

很多人写测试只考『输入一段正常数据』,结果空数组、只有一个数、全相同这些边界角落根本没考。偏偏 bug 最爱藏在这些角落:正常时跑得好好的,一遇到极端输入就崩或者算错。写考题一定要专门为空数据、单值、极端值各出一道边界题,把容易翻车的地方都覆盖到。

△ 02

测试跟着代码一起错(测的就是错逻辑)

如果你的考题答案是直接拿被测函数自己跑出来的结果当标准答案,那等于自己出题自己抄答案,函数算错了考题也跟着错,根本抓不出 bug。正确做法是用手算、查表、或另一种独立方式得到标准答案,再让函数去对。考题的答案必须独立于被测代码。

△ 03

改了代码却不重跑测试

单元测试的全部价值在于『一改代码就全部重考』。很多人写完一套考题就丢在那,后面改代码图省事不跑测试,等于护栏形同虚设。养成习惯:每次改完代码、提交之前,都把测试整套跑一遍,几秒钟的事,能帮你挡掉无数暗坑。最好接到自动化流程里,改一次自动跑一次。

△ 04

外部依赖没隔离,测试不稳定

如果你的函数里直接联网拉数据、读实时行情,那测试结果就会被网络和数据波动牵着走,今天过明天挂,你都搞不清是代码坏了还是数据变了。正确做法是把外部依赖用 mocker 模拟成固定的假数据,让测试只考你自己的逻辑,每次跑结果都一样、稳定可复现。

△ 05

量化代码干脆不写测试,bug 直接亏钱

很多散户觉得自己代码就几十行,写测试太麻烦,跑通一次就敢上实盘。可量化代码出错往往不报错、不吭声,指标算反、回测有 bug,数字照样漂亮,你照着下单亏的是真金白银,就像老蒋那样。越是真金白银驱动的代码,越值得花点时间出几道考题盯着它。

06 · STANDARD OPERATING PROCEDURE · 实战规则

给量化代码写单元测试的几条规矩

// 按这套规则走,你的执行力会立刻拉到中等机构水平

- 01 每个关键函数都配考题:正常用例 + 边界用例(空数据、单值、极端值)缺一不可
- 02 考题的标准答案要独立得来(手算、查表),绝不拿被测函数自己的输出当答案
- 03 每次改代码、提交前都把测试整套重跑一遍,最好接进自动化流程改一次跑一次
- 04 用 fixture 复用公共数据、用 parametrize 一道题套多组输入,少写重复测试代码
- 05 联网、实时行情这类外部依赖用 mocker 模拟成固定假数据,让测试稳定可复现

► **纪律 > 灵感:** 量化做久的人都明白,赚钱的不是某一次绝妙判断,而是把规则坚持执行 1000 次。打印这一页,贴在你电脑边。

07 · RECAP · 一页课件总结

把这节课带走的几件事

// 打印或截图这一页, 3 天后再看一遍效果最好

- 01 单元测试 = 给每个函数出一套答案已知的标准考题, 一改代码就全部重考, 谁错谁报红。
- 02 assert 断言写考题答案: 断定某事为真, 不真就报错; 一个测试可以写好几条 assert。
- 03 fixture 把公共的测试数据准备好放一处, 哪道考题要用就直接领用, 数据只写一遍。
- 04 parametrize 参数化把多组输入和答案列成表, 一道题自动各跑一遍, 覆盖面成倍扩大。
- 05 边界用例(空数据、单值、全相同、极端值)最容易藏 bug, 必须专门出题覆盖。
- 06 量化代码出错常常不吭声, bug 直接亏钱; 主动写测试, 改代码时考题替你盯着护栏。

08 · SELF-CHECK · 自测题

答不上来的回看视频对应段落

// 这几道题都没标准答案,目的是逼你自己组织语言讲清楚

Q01 用『考题』打比方,解释什么是单元测试,以及为什么一改代码就要把所有考题重跑一遍。

Q02 assert 断言是干什么的?写一句 assert 表示『2 加 3 应该等于 5』,说说它什么时候会报警。

Q03 为什么写测试只考正常情况不够?举几个最容易藏 bug 的边界用例,并说说老蒋的 bug 藏在哪。

Q04 fixture 和 parametrize 各帮你解决什么麻烦?为什么量化代码尤其值得写单元测试?

09 · NEXT STEP · 下一步

继续往前走

// 看完这节,下面是延伸方向 + 明天预告

推荐阅读 · FURTHER READING

// 听完今天的内容还想往深里挖的话

- ▶ `pytest` 官方文档(docs.pytest.org)– `assert`、`fixture`、`parametrize`、`mock` 等用法的权威手册,边查边学最快。
- ▶ Brian Okken 《Python Testing with pytest》(2022/2nd ed, Pragmatic Bookshelf)– 用 `pytest` 写测试的最佳入门书,例子由浅入深。
- ▶ Harry Percival 《Test-Driven Development with Python》(2017/O'Reilly)– 讲『先写测试再写代码』的测试驱动开发思路,实战感强。
- ▶ Marcos Lopez de Prado 《Advances in Financial Machine Learning》(2018/Wiley)– 量化大牛反复强调代码可靠性与回测陷阱,讲透为什么量化代码必须严谨。
- ▶ Martin Fowler 《Unit Test》(martinfowler.com)– 一篇把单元测试讲透彻的经典好文,适合理解测试的边界与价值。

NEXT LECTURE · 下一节预告

DAY 057 Git 协作 workflow

有了单元测试这道护栏,你改代码就踏实多了。这一节是 D53 到 D56 这一小批工程能力的收尾,凑成第 14 个合集。下一批我们继续往工程和策略深处走,逐步迈向真正的策略开发与回测,把前面学的数据、指标、测试,拼成一套能跑能验的完整交易系统。